COMPLETE SOURCE CODE EXPLANATION

Data Repositories & Database Layer

Line-by-Line Explanation of JDBC Persistence & Connection Pooling

Detailed examination of all 9 DAO Repositories and 2 Config classes. Explains PreparedStatement parameterized SQL, atomic transactions, ResultSet mapping, and Try-With-Resources.

Project: Restaurant Management System

Language: Java 17 / JavaFX / JDBC

Scope: Detailed Line-by-Line Breakdown

Table of Contents

CustomerRepository.java	src\main\java\com\restaurant\repository\CustomerRepository.java
InventoryRepository.java	src\main\java\com\restaurant\repository\InventoryRepository.java
MenuRepository.java	src\main\java\com\restaurant\repository\MenuRepository.java
OrderRepository.java	src\main\java\com\restaurant\repository\OrderRepository.java
PaymentRepository.java	src\main\java\com\restaurant\repository\PaymentRepository.java
ReservationRepository.java	src\main\java\com\restaurant\repository\ReservationRepository.java
SupplierRepository.java	src\main\java\com\restaurant\repository\SupplierRepository.java
TableRepository.java	src\main\java\com\restaurant\repository\TableRepository.java
UserRepository.java	src\main\java\com\restaurant\repository\UserRepository.java
AppConfig.java	src\main\java\com\restaurant\config\AppConfig.java
DatabaseConfig.java	src\main\java\com\restaurant\config\DatabaseConfig.java

CustomerRepository.java

Path: src\main\java\com\restaurant\repository\CustomerRepository.java | Total Lines: 133 | Logical Blocks: 10

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Data Access Object (DAO) for Customer. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–10 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.exception.DatabaseOperationException;
5   import com.restaurant.model.Customer;
6
7   import java.sql.*;
8   import java.util.ArrayList;
9   import java.util.List;
10
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11  public class CustomerRepository {
```

Architecture Role	Defines CustomerRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–13 Class Members & Implementation

```
12      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
13
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–13 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 14–29 Method: findById()

```
14      public Customer findById(int customerId) {
15          String sql = "SELECT * FROM customers WHERE customer_id = ?";
16          try (Connection conn = dbConfig.getConnection();
17               PreparedStatement stmt = conn.prepareStatement(sql)) {
18              stmt.setInt(1, customerId);
19              try (ResultSet rs = stmt.executeQuery()) {
20                  if (rs.next()) {
21                      return mapResultSetToCustomer(rs);
22                  }
23              }
24          } catch (SQLException e) {
25              throw new DatabaseOperationException("Error finding customer by ID: " + customerId, e);
26          }
27          return null;
28      }
29
```

Component	Method: findById()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 14–29 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 30–45 Method: findByPhone()

```
30      public Customer findByPhone(String phone) {
31          String sql = "SELECT * FROM customers WHERE phone = ?";
32          try (Connection conn = dbConfig.getConnection();
33               PreparedStatement stmt = conn.prepareStatement(sql)) {
34              stmt.setString(1, phone);
35              try (ResultSet rs = stmt.executeQuery()) {
36                  if (rs.next()) {
37                      return mapResultSetToCustomer(rs);
38                  }
39              }
40          } catch (SQLException e) {
41              throw new DatabaseOperationException("Error finding customer by phone: " + phone, e);
42          }
43          return null;
44      }
45
```

Component	Method: findByPhone()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 30–45 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 46–60` **Method: findAll()**

```
46      public List<Customer> findAll() {
47          List<Customer> customers = new ArrayList<>();
48          String sql = "SELECT * FROM customers ORDER BY total_spent DESC";
49          try (Connection conn = dbConfig.getConnection();
50               PreparedStatement stmt = conn.prepareStatement(sql);
51               ResultSet rs = stmt.executeQuery()) {
52              while (rs.next()) {
53                  customers.add(mapResultSetToCustomer(rs));
54              }
55          } catch (SQLException e) {
56              throw new DatabaseOperationException("Error fetching all customers", e);
57          }
58          return customers;
59      }
60
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 46–60 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 61–87` **Method: create()**

```
61       public boolean create(Customer customer) {
62           String sql = "INSERT INTO customers (name, phone, email, address, loyalty_points, total_visit
s, total_spent) VALUES (?, ?, ?, ?, ?, ?, ?)";
63           try (Connection conn = dbConfig.getConnection();
64                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
65               stmt.setString(1, customer.getName());
66               stmt.setString(2, customer.getPhone());
67               stmt.setString(3, customer.getEmail());
68               stmt.setString(4, customer.getAddress());
69               stmt.setInt(5, customer.getLoyaltyPoints());
70               stmt.setInt(6, customer.getTotalVisits());
71               stmt.setDouble(7, customer.getTotalSpent());
72
73               int affected = stmt.executeUpdate();
74               if (affected > 0) {
75                   try (ResultSet keys = stmt.getGeneratedKeys()) {
76                       if (keys.next()) {
77                           customer.setCustomerId(keys.getInt(1));
78                       }
79                   }
80                   return true;
81               }
82           } catch (SQLException e) {
83               throw new DatabaseOperationException("Error creating customer: " + customer.getName(), e);
84           }
85           return false;
86       }
87
```

Component	Method: create()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 61–87 (27 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–105 Method: update()

```
 88       public boolean update(Customer customer) {
 89           String sql = "UPDATE customers SET name = ?, phone = ?, email = ?, address = ?, loyalty_points
= ?, total_visits = ?, total_spent = ? WHERE customer_id = ?";
 90           try (Connection conn = dbConfig.getConnection();
 91                PreparedStatement stmt = conn.prepareStatement(sql)) {
 92               stmt.setString(1, customer.getName());
 93               stmt.setString(2, customer.getPhone());
 94               stmt.setString(3, customer.getEmail());
 95               stmt.setString(4, customer.getAddress());
 96               stmt.setInt(5, customer.getLoyaltyPoints());
 97               stmt.setInt(6, customer.getTotalVisits());
 98               stmt.setDouble(7, customer.getTotalSpent());
 99               stmt.setInt(8, customer.getCustomerId());
100               return stmt.executeUpdate() > 0;
101           } catch (SQLException e) {
102               throw new DatabaseOperationException("Error updating customer: " + customer.getCustomerId
(), e);
103           }
104       }
105
```

Component	Method: update()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 88–105 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 106–116 Method: delete()

```
106       public boolean delete(int customerId) {
107           String sql = "DELETE FROM customers WHERE customer_id = ?";
108           try (Connection conn = dbConfig.getConnection();
109                PreparedStatement stmt = conn.prepareStatement(sql)) {
110               stmt.setInt(1, customerId);
111               return stmt.executeUpdate() > 0;
112           } catch (SQLException e) {
113               throw new DatabaseOperationException("Error deleting customer: " + customerId, e);
114           }
115       }
116
```

Component	Method: delete()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 106–116 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 117–133` Method: mapResultSetToCustomer()

```
117       private Customer mapResultSetToCustomer(ResultSet rs) throws SQLException {
118           Customer c = new Customer();
119           c.setCustomerId(rs.getInt("customer_id"));
120           c.setName(rs.getString("name"));
121           c.setPhone(rs.getString("phone"));
122           c.setEmail(rs.getString("email"));
123           c.setAddress(rs.getString("address"));
124           c.setLoyaltyPoints(rs.getInt("loyalty_points"));
125           c.setTotalVisits(rs.getInt("total_visits"));
126           c.setTotalSpent(rs.getDouble("total_spent"));
127           Timestamp created = rs.getTimestamp("created_at");
128           if (created != null) c.setCreatedAt(created.toLocalDateTime());
129           Timestamp updated = rs.getTimestamp("updated_at");
130           if (updated != null) c.setUpdatedAt(updated.toLocalDateTime());
131           return c;
132       }
133   }
```

Component	Method: mapResultSetToCustomer()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 117–133 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InventoryRepository.java

Path: src\main\java\com\restaurant\repository\InventoryRepository.java | Total Lines: 136 | Logical Blocks: 10

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Data Access Object (DAO) for Inventory. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–10 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.exception.DatabaseOperationException;
5   import com.restaurant.model.InventoryItem;
6
7   import java.sql.*;
8   import java.util.ArrayList;
9   import java.util.List;
10
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11  public class InventoryRepository {
```

Architecture Role	Defines InventoryRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–13 Class Members & Implementation

```
12      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
13
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–13 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 14–30 Method: findAll()

```
14      public List<InventoryItem> findAll() {
15          List<InventoryItem> list = new ArrayList<>();
16          String sql = "SELECT i.*, s.name as supplier_name FROM inventory i " +
17                       "LEFT JOIN suppliers s ON i.supplier_id = s.supplier_id " +
18                       "ORDER BY i.quantity ASC";
19          try (Connection conn = dbConfig.getConnection();
20               PreparedStatement stmt = conn.prepareStatement(sql);
21               ResultSet rs = stmt.executeQuery()) {
22              while (rs.next()) {
23                  list.add(mapResultSetToInventoryItem(rs));
24              }
25          } catch (SQLException e) {
26              throw new DatabaseOperationException("Error fetching inventory items", e);
27          }
28          return list;
29      }
30
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 14–30 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–47 Method: findLowStockItems()

```
31      public List<InventoryItem> findLowStockItems() {
32          List<InventoryItem> list = new ArrayList<>();
33          String sql = "SELECT i.*, s.name as supplier_name FROM inventory i " +
34                       "LEFT JOIN suppliers s ON i.supplier_id = s.supplier_id " +
35                       "WHERE i.quantity <= i.minimum_stock ORDER BY i.quantity ASC";
36          try (Connection conn = dbConfig.getConnection();
37               PreparedStatement stmt = conn.prepareStatement(sql);
38               ResultSet rs = stmt.executeQuery()) {
39              while (rs.next()) {
40                  list.add(mapResultSetToInventoryItem(rs));
41              }
42          } catch (SQLException e) {
43              throw new DatabaseOperationException("Error fetching low-stock inventory items", e);
44          }
45          return list;
46      }
47
```

Component	Method: findLowStockItems()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 31–47 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 48–76 Method: create()

```
48       public boolean create(InventoryItem item) {
49           String sql = "INSERT INTO inventory (ingredient_name, category, quantity, unit, minimum_stock,
unit_cost, supplier_id) " +
50                        "VALUES (?, ?, ?, ?, ?, ?, ?)";
51           try (Connection conn = dbConfig.getConnection();
52                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
53               stmt.setString(1, item.getIngredientName());
54               stmt.setString(2, item.getCategory());
55               stmt.setDouble(3, item.getQuantity());
56               stmt.setString(4, item.getUnit());
57               stmt.setDouble(5, item.getMinimumStock());
58               stmt.setDouble(6, item.getUnitCost());
59               if (item.getSupplierId() != null) stmt.setInt(7, item.getSupplierId());
60               else stmt.setNull(7, Types.INTEGER);
61
62               int affected = stmt.executeUpdate();
63               if (affected > 0) {
64                   try (ResultSet keys = stmt.getGeneratedKeys()) {
65                       if (keys.next()) {
66                           item.setInventoryId(keys.getInt(1));
67                       }
68                   }
69                   return true;
70               }
71           } catch (SQLException e) {
72               throw new DatabaseOperationException("Error creating inventory item: " + item.getIngredien
tName(), e);
73           }
74           return false;
75       }
76
```

Component	Method: create()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 48–76 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 77–88 Method: updateStock()

```
77        public boolean updateStock(int inventoryId, double newQuantity) {
78            String sql = "UPDATE inventory SET quantity = ? WHERE inventory_id = ?";
79            try (Connection conn = dbConfig.getConnection();
80                 PreparedStatement stmt = conn.prepareStatement(sql)) {
81                stmt.setDouble(1, newQuantity);
82                stmt.setInt(2, inventoryId);
83                return stmt.executeUpdate() > 0;
84            } catch (SQLException e) {
85                throw new DatabaseOperationException("Error updating inventory stock for ID: " + inventory
Id, e);
86            }
87        }
88
```

Component	Method: updateStock()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 77–88 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 89–108 Method: update()

```
89        public boolean update(InventoryItem item) {
90            String sql = "UPDATE inventory SET ingredient_name = ?, category = ?, quantity = ?, unit = ?,
minimum_stock = ?, unit_cost = ?, supplier_id = ? " +
91                         "WHERE inventory_id = ?";
92            try (Connection conn = dbConfig.getConnection();
93                 PreparedStatement stmt = conn.prepareStatement(sql)) {
94                stmt.setString(1, item.getIngredientName());
95                stmt.setString(2, item.getCategory());
96                stmt.setDouble(3, item.getQuantity());
97                stmt.setString(4, item.getUnit());
98                stmt.setDouble(5, item.getMinimumStock());
99                stmt.setDouble(6, item.getUnitCost());
100               if (item.getSupplierId() != null) stmt.setInt(7, item.getSupplierId());
101               else stmt.setNull(7, Types.INTEGER);
102               stmt.setInt(8, item.getInventoryId());
103               return stmt.executeUpdate() > 0;
104           } catch (SQLException e) {
105               throw new DatabaseOperationException("Error updating inventory item: " + item.getInventory
Id(), e);
106           }
107       }
108
```

Component	Method: update()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 89–108 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 109–119 Method: delete()

```
109       public boolean delete(int inventoryId) {
110           String sql = "DELETE FROM inventory WHERE inventory_id = ?";
111           try (Connection conn = dbConfig.getConnection();
112                PreparedStatement stmt = conn.prepareStatement(sql)) {
113               stmt.setInt(1, inventoryId);
114               return stmt.executeUpdate() > 0;
115           } catch (SQLException e) {
116               throw new DatabaseOperationException("Error deleting inventory item: " + inventoryId, e);
117           }
118       }
119
```

Component	Method: delete()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 109–119 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 120–136 Method: mapResultSetToInventoryItem()

```
120       private InventoryItem mapResultSetToInventoryItem(ResultSet rs) throws SQLException {
121           InventoryItem item = new InventoryItem();
122           item.setInventoryId(rs.getInt("inventory_id"));
123           item.setIngredientName(rs.getString("ingredient_name"));
124           item.setCategory(rs.getString("category"));
125           item.setQuantity(rs.getDouble("quantity"));
126           item.setUnit(rs.getString("unit"));
127           item.setMinimumStock(rs.getDouble("minimum_stock"));
128           item.setUnitCost(rs.getDouble("unit_cost"));
129           int supId = rs.getInt("supplier_id");
130           if (!rs.wasNull()) item.setSupplierId(supId);
131           item.setSupplierName(rs.getString("supplier_name"));
132           Timestamp ts = rs.getTimestamp("last_updated");
133           if (ts != null) item.setLastUpdated(ts.toLocalDateTime());
134           return item;
135       }
136   }
```

Component	Method: mapResultSetToInventoryItem()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 120–136 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

MenuRepository.java

Path: src\main\java\com\restaurant\repository\MenuRepository.java | Total Lines: 138 | Logical Blocks: 10

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Data Access Object (DAO) for Menu. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–11 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.exception.DatabaseOperationException;
5   import com.restaurant.model.MenuCategory;
6   import com.restaurant.model.MenuItem;
7
8   import java.sql.*;
9   import java.util.ArrayList;
10  import java.util.List;
11
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 11. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 12–12 Class Declaration & Heritage

```
12  public class MenuRepository {
```

Architecture Role	Defines MenuRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 12, allowing cross-package invocation.

Lines 13–14 Class Members & Implementation

```
13      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
14
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 13–14 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 15–34 Method: findAllCategories()

```
15      public List<MenuCategory> findAllCategories() {
16          List<MenuCategory> categories = new ArrayList<>();
17          String sql = "SELECT * FROM menu_categories ORDER BY display_order ASC, name ASC";
18          try (Connection conn = dbConfig.getConnection();
19               PreparedStatement stmt = conn.prepareStatement(sql);
20               ResultSet rs = stmt.executeQuery()) {
21              while (rs.next()) {
22                  categories.add(new MenuCategory(
23                          rs.getInt("category_id"),
24                          rs.getString("name"),
25                          rs.getString("description"),
26                          rs.getInt("display_order")
27                  ));
28              }
29          } catch (SQLException e) {
30              throw new DatabaseOperationException("Error fetching menu categories", e);
31          }
32          return categories;
33      }
34
```

Component	Method: findAllCategories()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 15–34 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–51 Method: findAllItems()

```
35      public List<MenuItem> findAllItems() {
36          List<MenuItem> items = new ArrayList<>();
37          String sql = "SELECT m.*, c.name as category_name FROM menu_items m " +
38                       "JOIN menu_categories c ON m.category_id = c.category_id " +
39                       "ORDER BY c.display_order, m.name";
40          try (Connection conn = dbConfig.getConnection();
41               PreparedStatement stmt = conn.prepareStatement(sql);
42               ResultSet rs = stmt.executeQuery()) {
43              while (rs.next()) {
44                  items.add(mapResultSetToMenuItem(rs));
45              }
46          } catch (SQLException e) {
47              throw new DatabaseOperationException("Error fetching menu items", e);
48          }
49          return items;
50      }
51
```

Component	Method: findAllItems()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 35–51 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 52–69 Method: findItemById()

```
52      public MenuItem findItemById(int itemId) {
53          String sql = "SELECT m.*, c.name as category_name FROM menu_items m " +
54                       "JOIN menu_categories c ON m.category_id = c.category_id " +
55                       "WHERE m.item_id = ?";
56          try (Connection conn = dbConfig.getConnection();
57               PreparedStatement stmt = conn.prepareStatement(sql)) {
58              stmt.setInt(1, itemId);
59              try (ResultSet rs = stmt.executeQuery()) {
60                  if (rs.next()) {
61                      return mapResultSetToMenuItem(rs);
62                  }
63              }
64          } catch (SQLException e) {
65              throw new DatabaseOperationException("Error finding menu item by ID: " + itemId, e);
66          }
67          return null;
68      }
69
```

Component	Method: findItemById()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 52–69 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 70–95` **Method: createItem()**

```
70       public boolean createItem(MenuItem item) {
71           String sql = "INSERT INTO menu_items (category_id, name, description, price, preparation_time,
is_available) VALUES (?, ?, ?, ?, ?, ?)";
72           try (Connection conn = dbConfig.getConnection();
73                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
74               stmt.setInt(1, item.getCategoryId());
75               stmt.setString(2, item.getName());
76               stmt.setString(3, item.getDescription());
77               stmt.setDouble(4, item.getPrice());
78               stmt.setInt(5, item.getPreparationTime());
79               stmt.setBoolean(6, item.isAvailable());
80
81               int affected = stmt.executeUpdate();
82               if (affected > 0) {
83                   try (ResultSet keys = stmt.getGeneratedKeys()) {
84                       if (keys.next()) {
85                           item.setItemId(keys.getInt(1));
86                       }
87                   }
88                   return true;
89               }
90           } catch (SQLException e) {
91               throw new DatabaseOperationException("Error creating menu item: " + item.getName(), e);
92           }
93           return false;
94       }
95
```

Component	Method: createItem()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 70–95 (26 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 96–112 Method: updateItem()

```
 96       public boolean updateItem(MenuItem item) {
 97           String sql = "UPDATE menu_items SET category_id = ?, name = ?, description = ?, price = ?, pre
paration_time = ?, is_available = ? WHERE item_id = ?";
 98           try (Connection conn = dbConfig.getConnection();
 99                PreparedStatement stmt = conn.prepareStatement(sql)) {
100               stmt.setInt(1, item.getCategoryId());
101               stmt.setString(2, item.getName());
102               stmt.setString(3, item.getDescription());
103               stmt.setDouble(4, item.getPrice());
104               stmt.setInt(5, item.getPreparationTime());
105               stmt.setBoolean(6, item.isAvailable());
106               stmt.setInt(7, item.getItemId());
107               return stmt.executeUpdate() > 0;
108           } catch (SQLException e) {
109               throw new DatabaseOperationException("Error updating menu item: " + item.getItemId(), e);
110           }
111       }
112
```

Component	Method: updateItem()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 96–112 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 113–123 Method: deleteItem()

```
113       public boolean deleteItem(int itemId) {
114           String sql = "DELETE FROM menu_items WHERE item_id = ?";
115           try (Connection conn = dbConfig.getConnection();
116                PreparedStatement stmt = conn.prepareStatement(sql)) {
117               stmt.setInt(1, itemId);
118               return stmt.executeUpdate() > 0;
119           } catch (SQLException e) {
120               throw new DatabaseOperationException("Error deleting menu item: " + itemId, e);
121           }
122       }
123
```

Component	Method: deleteItem()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 113–123 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 124–138` Method: mapResultSetToMenuItem()

```
124       private MenuItem mapResultSetToMenuItem(ResultSet rs) throws SQLException {
125           MenuItem item = new MenuItem(
126                   rs.getInt("item_id"),
127                   rs.getInt("category_id"),
128                   rs.getString("name"),
129                   rs.getString("description"),
130                   rs.getDouble("price"),
131                   rs.getInt("preparation_time"),
132                   rs.getBoolean("is_available")
133           );
134           item.setCategoryName(rs.getString("category_name"));
135           item.setImagePath(rs.getString("image_path"));
136           return item;
137       }
138   }
```

Component	Method: mapResultSetToMenuItem()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 124–138 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

OrderRepository.java

Path: src\main\java\com\restaurant\repository\OrderRepository.java | Total Lines: 218 | Logical Blocks: 11

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Data Access Object (DAO) for Order. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–12 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.enums.OrderStatus;
5   import com.restaurant.exception.DatabaseOperationException;
6   import com.restaurant.model.Order;
7   import com.restaurant.model.OrderItem;
8
9   import java.sql.*;
10  import java.util.ArrayList;
11  import java.util.List;
12
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 12. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 13–13 Class Declaration & Heritage

```
13  public class OrderRepository {
```

Architecture Role	Defines OrderRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 13, allowing cross-package invocation.

Lines 14–15 Class Members & Implementation

```
14      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
15
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 14–15 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 16–38` **Method: findById()**

```
16        public Order findById(int orderId) {
17            String sql = "SELECT o.*, c.name as customer_name, t.table_number, u.full_name as employee_nam
e " +
18                         "FROM orders o " +
19                         "LEFT JOIN customers c ON o.customer_id = c.customer_id " +
20                         "LEFT JOIN restaurant_tables t ON o.table_id = t.table_id " +
21                         "JOIN users u ON o.employee_id = u.user_id " +
22                         "WHERE o.order_id = ?";
23            try (Connection conn = dbConfig.getConnection();
24                 PreparedStatement stmt = conn.prepareStatement(sql)) {
25                stmt.setInt(1, orderId);
26                try (ResultSet rs = stmt.executeQuery()) {
27                    if (rs.next()) {
28                        Order order = mapResultSetToOrder(rs);
29                        order.setItems(findItemsByOrderId(order.getOrderId()));
30                        return order;
31                    }
32                }
33            } catch (SQLException e) {
34                throw new DatabaseOperationException("Error finding order by ID: " + orderId, e);
35            }
36            return null;
37        }
38
```

Component	Method: findById()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 16–38 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 39–59` **Method: findAll()**

```
39        public List<Order> findAll() {
40            List<Order> orders = new ArrayList<>();
41            String sql = "SELECT o.*, c.name as customer_name, t.table_number, u.full_name as employee_nam
e " +
42                         "FROM orders o " +
43                         "LEFT JOIN customers c ON o.customer_id = c.customer_id " +
44                         "LEFT JOIN restaurant_tables t ON o.table_id = t.table_id " +
45                         "JOIN users u ON o.employee_id = u.user_id " +
46                         "ORDER BY o.order_date_time DESC";
47            try (Connection conn = dbConfig.getConnection();
48                 PreparedStatement stmt = conn.prepareStatement(sql);
49                 ResultSet rs = stmt.executeQuery()) {
50                while (rs.next()) {
51                    Order order = mapResultSetToOrder(rs);
52                    orders.add(order);
53                }
54            } catch (SQLException e) {
55                throw new DatabaseOperationException("Error fetching orders", e);
56            }
57            return orders;
58        }
59
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 39–59 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 60–82 Method: findActiveKitchenOrders()

```
60        public List<Order> findActiveKitchenOrders() {
61            List<Order> orders = new ArrayList<>();
62            String sql = "SELECT o.*, c.name as customer_name, t.table_number, u.full_name as employee_nam
e " +
63                         "FROM orders o " +
64                         "LEFT JOIN customers c ON o.customer_id = c.customer_id " +
65                         "LEFT JOIN restaurant_tables t ON o.table_id = t.table_id " +
66                         "JOIN users u ON o.employee_id = u.user_id " +
67                         "WHERE o.status IN ('CONFIRMED', 'PREPARING', 'READY') " +
68                         "ORDER BY o.order_date_time ASC";
69            try (Connection conn = dbConfig.getConnection();
70                 PreparedStatement stmt = conn.prepareStatement(sql);
71                 ResultSet rs = stmt.executeQuery()) {
72                while (rs.next()) {
73                    Order order = mapResultSetToOrder(rs);
74                    order.setItems(findItemsByOrderId(order.getOrderId()));
75                    orders.add(order);
76                }
77            } catch (SQLException e) {
78                throw new DatabaseOperationException("Error fetching kitchen orders", e);
79            }
80            return orders;
81        }
82
```

Component	Method: findActiveKitchenOrders()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 60–82 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 83–110 Method: findItemsByOrderId()

```
83      public List<OrderItem> findItemsByOrderId(int orderId) {
84          List<OrderItem> items = new ArrayList<>();
85          String sql = "SELECT oi.*, m.name as item_name FROM order_items oi " +
86                       "JOIN menu_items m ON oi.item_id = m.item_id WHERE oi.order_id = ?";
87          try (Connection conn = dbConfig.getConnection();
88               PreparedStatement stmt = conn.prepareStatement(sql)) {
89              stmt.setInt(1, orderId);
90              try (ResultSet rs = stmt.executeQuery()) {
91                  while (rs.next()) {
92                      OrderItem item = new OrderItem();
93                      item.setOrderItemId(rs.getInt("order_item_id"));
94                      item.setOrderId(rs.getInt("order_id"));
95                      item.setItemId(rs.getInt("item_id"));
96                      item.setItemName(rs.getString("item_name"));
97                      item.setQuantity(rs.getInt("quantity"));
98                      item.setUnitPrice(rs.getDouble("unit_price"));
99                      item.setSpecialInstructions(rs.getString("special_instructions"));
100                     item.setSubtotal(rs.getDouble("subtotal"));
101                     item.setStatus(rs.getString("status"));
102                     items.add(item);
103                 }
104             }
105         } catch (SQLException e) {
106             throw new DatabaseOperationException("Error fetching order items for order: " + orderId,
e);
107         }
108         return items;
109     }
110
```

Component	Method: findItemsByOrderId()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 83–110 (28 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 111–163 **Method: createOrderWithItems()**

```
111       public boolean createOrderWithItems(Order order) {
112           String insertOrderSql = "INSERT INTO orders (order_number, customer_id, table_id, employee_id,
order_date_time, status, subtotal, discount_amount, tax_amount, service_charge, total_amount, special_notes)
" +
113                                   "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
114           String insertItemSql = "INSERT INTO order_items (order_id, item_id, quantity, unit_price, spec
ial_instructions, subtotal, status) VALUES (?, ?, ?, ?, ?, ?, ?)";
115
116           Connection conn = null;
117           try {
118               conn = dbConfig.getConnection();
119               conn.setAutoCommit(false); // Transaction begins
120
121               try (PreparedStatement orderStmt = conn.prepareStatement(insertOrderSql, Statement.RETURN_
GENERATED_KEYS)) {
122                   orderStmt.setString(1, order.getOrderNumber());
123                   if (order.getCustomerId() != null) orderStmt.setInt(2, order.getCustomerId());
124                   else orderStmt.setNull(2, Types.INTEGER);
125
126                   if (order.getTableId() != null) orderStmt.setInt(3, order.getTableId());
127                   else orderStmt.setNull(3, Types.INTEGER);
128
129                   orderStmt.setInt(4, order.getEmployeeId());
130                   orderStmt.setTimestamp(5, Timestamp.valueOf(order.getOrderDateTime()));
131                   orderStmt.setString(6, order.getStatus().name());
132                   orderStmt.setDouble(7, order.getSubtotal());
133                   orderStmt.setDouble(8, order.getDiscountAmount());
134                   orderStmt.setDouble(9, order.getTaxAmount());
135                   orderStmt.setDouble(10, order.getServiceCharge());
136                   orderStmt.setDouble(11, order.getTotalAmount());
137                   orderStmt.setString(12, order.getSpecialNotes());
138
139                   int affected = orderStmt.executeUpdate();
140                   if (affected == 0) {
141                       conn.rollback();
142                       return false;
143                   }
144
145                   try (ResultSet keys = orderStmt.getGeneratedKeys()) {
146                       if (keys.next()) {
147                           order.setOrderId(keys.getInt(1));
148                       }
149                   }
150               }
151
152               // Insert line items
153               try (PreparedStatement itemStmt = conn.prepareStatement(insertItemSql)) {
154                   for (OrderItem item : order.getItems()) {
155                       itemStmt.setInt(1, order.getOrderId());
156                       itemStmt.setInt(2, item.getItemId());
157                       itemStmt.setInt(3, item.getQuantity());
158                       itemStmt.setDouble(4, item.getUnitPrice());
159                       itemStmt.setString(5, item.getSpecialInstructions());
160                       itemStmt.setDouble(6, item.getSubtotal());
161                       itemStmt.setString(7, item.getStatus() != null ? item.getStatus() : "PENDING");
162                       itemStmt.addBatch();
163                   }
```

Component	Method: createOrderWithItems()

Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 111–163 (53 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 164–180` Class Members & Continuation

```
164                     itemStmt.executeBatch();
165                 }
166 
167                 conn.commit(); // Transaction commits
168                 return true;
169             } catch (SQLException e) {
170                 if (conn != null) {
171                     try { conn.rollback(); } catch (SQLException ex) { /* ignored */ }
172                 }
173                 throw new DatabaseOperationException("Error saving order transaction: " + order.getOrderNu
mber(), e);
174             } finally {
175                 if (conn != null) {
176                     try { conn.setAutoCommit(true); conn.close(); } catch (SQLException e) { /* ignored */
}
177                 }
178             }
179         }
180 
```

Component	Class Members & Continuation
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 164–180 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 181–192` Method: updateStatus()

```
181       public boolean updateStatus(int orderId, OrderStatus status) {
182           String sql = "UPDATE orders SET status = ? WHERE order_id = ?";
183           try (Connection conn = dbConfig.getConnection();
184                PreparedStatement stmt = conn.prepareStatement(sql)) {
185               stmt.setString(1, status.name());
186               stmt.setInt(2, orderId);
187               return stmt.executeUpdate() > 0;
188           } catch (SQLException e) {
189               throw new DatabaseOperationException("Error updating status for order: " + orderId, e);
190           }
191       }
192
```

Component	Method: updateStatus()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 181–192 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 193–218` **Method: mapResultSetToOrder()**

```
193       private Order mapResultSetToOrder(ResultSet rs) throws SQLException {
194           Order o = new Order();
195           o.setOrderId(rs.getInt("order_id"));
196           o.setOrderNumber(rs.getString("order_number"));
197           int custId = rs.getInt("customer_id");
198           if (!rs.wasNull()) o.setCustomerId(custId);
199           o.setCustomerName(rs.getString("customer_name"));
200
201           int tableId = rs.getInt("table_id");
202           if (!rs.wasNull()) o.setTableId(tableId);
203           o.setTableNumber(rs.getString("table_number"));
204
205           o.setEmployeeId(rs.getInt("employee_id"));
206           o.setEmployeeName(rs.getString("employee_name"));
207           Timestamp ts = rs.getTimestamp("order_date_time");
208           if (ts != null) o.setOrderDateTime(ts.toLocalDateTime());
209           o.setStatus(OrderStatus.valueOf(rs.getString("status")));
210           o.setSubtotal(rs.getDouble("subtotal"));
211           o.setDiscountAmount(rs.getDouble("discount_amount"));
212           o.setTaxAmount(rs.getDouble("tax_amount"));
213           o.setServiceCharge(rs.getDouble("service_charge"));
214           o.setTotalAmount(rs.getDouble("total_amount"));
215           o.setSpecialNotes(rs.getString("special_notes"));
216           return o;
217       }
218   }
```

Component	Method: mapResultSetToOrder()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 193–218 (26 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PaymentRepository.java

Path: src\main\java\com\restaurant\repository\PaymentRepository.java | Total Lines: 96 | Logical Blocks: 7

📘 COMPONENT OVERVIEW & RESPONSIBILITY

Data Access Object (DAO) for Payment. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–12 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.enums.PaymentMethod;
5   import com.restaurant.enums.PaymentStatus;
6   import com.restaurant.exception.DatabaseOperationException;
7   import com.restaurant.model.*;
8
9   import java.sql.*;
10  import java.util.ArrayList;
11  import java.util.List;
12
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 12. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 13–13 Class Declaration & Heritage

```
13  public class PaymentRepository {
```

Architecture Role	Defines PaymentRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 13, allowing cross-package invocation.

Lines 14–15 Class Members & Implementation

```
14      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
15
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 14–15 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 16–44` **Method: recordPayment()**

```
16       public boolean recordPayment(Payment payment) {
17           String sql = "INSERT INTO payments (order_id, invoice_number, amount, payment_method, payment_
date_time, transaction_reference, status, processed_by) " +
18                        "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
19           try (Connection conn = dbConfig.getConnection();
20                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
21               stmt.setInt(1, payment.getOrderId());
22               stmt.setString(2, payment.getInvoiceNumber());
23               stmt.setDouble(3, payment.getAmount());
24               stmt.setString(4, payment.getPaymentMethod().name());
25               stmt.setTimestamp(5, Timestamp.valueOf(payment.getPaymentDateTime()));
26               stmt.setString(6, payment.getTransactionReference());
27               stmt.setString(7, payment.getStatus().name());
28               stmt.setInt(8, payment.getProcessedBy());
29
30               int affected = stmt.executeUpdate();
31               if (affected > 0) {
32                   try (ResultSet keys = stmt.getGeneratedKeys()) {
33                       if (keys.next()) {
34                           payment.setPaymentId(keys.getInt(1));
35                       }
36                   }
37                   return true;
38               }
39           } catch (SQLException e) {
40               throw new DatabaseOperationException("Error recording payment transaction: " + payment.get
InvoiceNumber(), e);
41           }
42           return false;
43       }
44
```

Component	Method: recordPayment()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 16–44 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 45–60 Method: findByOrderId()

```
45      public Payment findByOrderId(int orderId) {
46          String sql = "SELECT * FROM payments WHERE order_id = ?";
47          try (Connection conn = dbConfig.getConnection();
48               PreparedStatement stmt = conn.prepareStatement(sql)) {
49              stmt.setInt(1, orderId);
50              try (ResultSet rs = stmt.executeQuery()) {
51                  if (rs.next()) {
52                      return mapResultSetToPayment(rs);
53                  }
54              }
55          } catch (SQLException e) {
56              throw new DatabaseOperationException("Error finding payment for order ID: " + orderId, e);
57          }
58          return null;
59      }
60
```

Component	Method: findByOrderId()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 45–60 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 61–75 Method: findAll()

```
61      public List<Payment> findAll() {
62          List<Payment> list = new ArrayList<>();
63          String sql = "SELECT * FROM payments ORDER BY payment_date_time DESC";
64          try (Connection conn = dbConfig.getConnection();
65               PreparedStatement stmt = conn.prepareStatement(sql);
66               ResultSet rs = stmt.executeQuery()) {
67              while (rs.next()) {
68                  list.add(mapResultSetToPayment(rs));
69              }
70          } catch (SQLException e) {
71              throw new DatabaseOperationException("Error fetching payments", e);
72          }
73          return list;
74      }
75
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 61–75 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 76–96 Method: mapResultSetToPayment()

```
76      private Payment mapResultSetToPayment(ResultSet rs) throws SQLException {
77          PaymentMethod method = PaymentMethod.valueOf(rs.getString("payment_method"));
78          Payment payment;
79          switch (method) {
80              case CASH -> payment = new CashPayment();
81              case UPI -> payment = new UPIPayment();
82              case CREDIT_CARD, DEBIT_CARD -> payment = new CardPayment();
83              default -> payment = new CashPayment();
84          }
85          payment.setPaymentId(rs.getInt("payment_id"));
86          payment.setOrderId(rs.getInt("order_id"));
87          payment.setInvoiceNumber(rs.getString("invoice_number"));
88          payment.setAmount(rs.getDouble("amount"));
89          payment.setPaymentMethod(method);
90          payment.setPaymentDateTime(rs.getTimestamp("payment_date_time").toLocalDateTime());
91          payment.setTransactionReference(rs.getString("transaction_reference"));
92          payment.setStatus(PaymentStatus.valueOf(rs.getString("status")));
93          payment.setProcessedBy(rs.getInt("processed_by"));
94          return payment;
95      }
96  }
```

Component	Method: mapResultSetToPayment()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 76–96 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

ReservationRepository.java

Path: src\main\java\com\restaurant\repository\ReservationRepository.java | Total Lines: 115 | Logical Blocks: 8

📘 COMPONENT OVERVIEW & RESPONSIBILITY

Data Access Object (DAO) for Reservation. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–13 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.enums.ReservationStatus;
5   import com.restaurant.exception.DatabaseOperationException;
6   import com.restaurant.model.Reservation;
7
8   import java.sql.*;
9   import java.time.LocalDate;
10  import java.time.LocalTime;
11  import java.util.ArrayList;
12  import java.util.List;
13
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 13. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 14–14 Class Declaration & Heritage

```
14  public class ReservationRepository {
```

Architecture Role	Defines ReservationRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 14, allowing cross-package invocation.

Lines 15–16 Class Members & Implementation

```
15      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
16
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 15–16 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 17–35` **Method: findAll()**

```
17      public List<Reservation> findAll() {
18          List<Reservation> list = new ArrayList<>();
19          String sql = "SELECT r.*, c.name as customer_name, c.phone as customer_phone, t.table_number "
+
20                       "FROM reservations r " +
21                       "JOIN customers c ON r.customer_id = c.customer_id " +
22                       "JOIN restaurant_tables t ON r.table_id = t.table_id " +
23                       "ORDER BY r.reservation_date DESC, r.reservation_time ASC";
24          try (Connection conn = dbConfig.getConnection();
25               PreparedStatement stmt = conn.prepareStatement(sql);
26               ResultSet rs = stmt.executeQuery()) {
27              while (rs.next()) {
28                  list.add(mapResultSetToReservation(rs));
29              }
30          } catch (SQLException e) {
31              throw new DatabaseOperationException("Error fetching reservations", e);
32          }
33          return list;
34      }
35
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 17–35 (19 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 36–59` **Method: hasConflict()**

```
36        public boolean hasConflict(int tableId, LocalDate date, LocalTime time, int excludeReservationId)
{
37            String sql = "SELECT COUNT(*) FROM reservations " +
38                         "WHERE table_id = ? AND reservation_date = ? " +
39                         "AND status IN ('PENDING', 'CONFIRMED') " +
40                         "AND reservation_id != ? " +
41                         "AND (ABS(TIME_TO_SEC(TIMEDIFF(reservation_time, ?))) < 7200)"; // 2-hour window
conflict check
42            try (Connection conn = dbConfig.getConnection();
43                 PreparedStatement stmt = conn.prepareStatement(sql)) {
44                stmt.setInt(1, tableId);
45                stmt.setDate(2, Date.valueOf(date));
46                stmt.setInt(3, excludeReservationId);
47                stmt.setTime(4, Time.valueOf(time));
48
49                try (ResultSet rs = stmt.executeQuery()) {
50                    if (rs.next()) {
51                        return rs.getInt(1) > 0;
52                    }
53                }
54            } catch (SQLException e) {
55                throw new DatabaseOperationException("Error checking reservation conflict", e);
56            }
57            return false;
58        }
59
```

Component	Method: hasConflict()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 36–59 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 60–87` **Method: create()**

```
60       public boolean create(Reservation r) {
61           String sql = "INSERT INTO reservations (customer_id, table_id, reservation_date, reservation_t
ime, guest_count, special_request, status) " +
62                        "VALUES (?, ?, ?, ?, ?, ?, ?)";
63           try (Connection conn = dbConfig.getConnection();
64                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
65               stmt.setInt(1, r.getCustomerId());
66               stmt.setInt(2, r.getTableId());
67               stmt.setDate(3, Date.valueOf(r.getReservationDate()));
68               stmt.setTime(4, Time.valueOf(r.getReservationTime()));
69               stmt.setInt(5, r.getGuestCount());
70               stmt.setString(6, r.getSpecialRequest());
71               stmt.setString(7, r.getStatus().name());
72
73               int affected = stmt.executeUpdate();
74               if (affected > 0) {
75                   try (ResultSet keys = stmt.getGeneratedKeys()) {
76                       if (keys.next()) {
77                           r.setReservationId(keys.getInt(1));
78                       }
79                   }
80                   return true;
81               }
82           } catch (SQLException e) {
83               throw new DatabaseOperationException("Error creating reservation", e);
84           }
85           return false;
86       }
87
```

Component	Method: create()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 60–87 (28 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–99 Method: updateStatus()

```
88        public boolean updateStatus(int reservationId, ReservationStatus status) {
89            String sql = "UPDATE reservations SET status = ? WHERE reservation_id = ?";
90            try (Connection conn = dbConfig.getConnection();
91                 PreparedStatement stmt = conn.prepareStatement(sql)) {
92                stmt.setString(1, status.name());
93                stmt.setInt(2, reservationId);
94                return stmt.executeUpdate() > 0;
95            } catch (SQLException e) {
96                throw new DatabaseOperationException("Error updating reservation status: " + reservationI
d, e);
97            }
98        }
99
```

Component	Method: updateStatus()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 88–99 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 100–115 Method: mapResultSetToReservation()

```
100       private Reservation mapResultSetToReservation(ResultSet rs) throws SQLException {
101           Reservation r = new Reservation();
102           r.setReservationId(rs.getInt("reservation_id"));
103           r.setCustomerId(rs.getInt("customer_id"));
104           r.setCustomerName(rs.getString("customer_name"));
105           r.setCustomerPhone(rs.getString("customer_phone"));
106           r.setTableId(rs.getInt("table_id"));
107           r.setTableNumber(rs.getString("table_number"));
108           r.setReservationDate(rs.getDate("reservation_date").toLocalDate());
109           r.setReservationTime(rs.getTime("reservation_time").toLocalTime());
110           r.setGuestCount(rs.getInt("guest_count"));
111           r.setSpecialRequest(rs.getString("special_request"));
112           r.setStatus(ReservationStatus.valueOf(rs.getString("status")));
113           return r;
114       }
115   }
```

Component	Method: mapResultSetToReservation()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 100–115 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

SupplierRepository.java

Path: src\main\java\com\restaurant\repository\SupplierRepository.java | Total Lines: 112 | Logical Blocks: 9

📘 COMPONENT OVERVIEW & RESPONSIBILITY

Data Access Object (DAO) for Supplier. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–10 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.exception.DatabaseOperationException;
5   import com.restaurant.model.Supplier;
6
7   import java.sql.*;
8   import java.util.ArrayList;
9   import java.util.List;
10
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11  public class SupplierRepository {
```

Architecture Role	Defines SupplierRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–13 Class Members & Implementation

```
12      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
13
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–13 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 14–28 Method: findAll()

```
14      public List<Supplier> findAll() {
15          List<Supplier> list = new ArrayList<>();
16          String sql = "SELECT * FROM suppliers ORDER BY name ASC";
17          try (Connection conn = dbConfig.getConnection();
18               PreparedStatement stmt = conn.prepareStatement(sql);
19               ResultSet rs = stmt.executeQuery()) {
20              while (rs.next()) {
21                  list.add(mapResultSetToSupplier(rs));
22              }
23          } catch (SQLException e) {
24              throw new DatabaseOperationException("Error fetching suppliers", e);
25          }
26          return list;
27      }
28
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 14–28 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–44 Method: findById()

```
29      public Supplier findById(int supplierId) {
30          String sql = "SELECT * FROM suppliers WHERE supplier_id = ?";
31          try (Connection conn = dbConfig.getConnection();
32               PreparedStatement stmt = conn.prepareStatement(sql)) {
33              stmt.setInt(1, supplierId);
34              try (ResultSet rs = stmt.executeQuery()) {
35                  if (rs.next()) {
36                      return mapResultSetToSupplier(rs);
37                  }
38              }
39          } catch (SQLException e) {
40              throw new DatabaseOperationException("Error finding supplier by ID: " + supplierId, e);
41          }
42          return null;
43      }
44
```

Component	Method: findById()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 29–44 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 45–70` **Method: create()**

```
45       public boolean create(Supplier s) {
46           String sql = "INSERT INTO suppliers (name, contact_person, phone, email, address, supplied_cat
egory) VALUES (?, ?, ?, ?, ?, ?)";
47           try (Connection conn = dbConfig.getConnection();
48                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
49               stmt.setString(1, s.getName());
50               stmt.setString(2, s.getContactPerson());
51               stmt.setString(3, s.getPhone());
52               stmt.setString(4, s.getEmail());
53               stmt.setString(5, s.getAddress());
54               stmt.setString(6, s.getSuppliedCategory());
55
56               int affected = stmt.executeUpdate();
57               if (affected > 0) {
58                   try (ResultSet keys = stmt.getGeneratedKeys()) {
59                       if (keys.next()) {
60                           s.setSupplierId(keys.getInt(1));
61                       }
62                   }
63                   return true;
64               }
65           } catch (SQLException e) {
66               throw new DatabaseOperationException("Error creating supplier: " + s.getName(), e);
67           }
68           return false;
69       }
70
```

Component	Method: create()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 45–70 (26 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 71–87 Method: update()

```
71       public boolean update(Supplier s) {
72           String sql = "UPDATE suppliers SET name = ?, contact_person = ?, phone = ?, email = ?, address
= ?, supplied_category = ? WHERE supplier_id = ?";
73           try (Connection conn = dbConfig.getConnection();
74                PreparedStatement stmt = conn.prepareStatement(sql)) {
75               stmt.setString(1, s.getName());
76               stmt.setString(2, s.getContactPerson());
77               stmt.setString(3, s.getPhone());
78               stmt.setString(4, s.getEmail());
79               stmt.setString(5, s.getAddress());
80               stmt.setString(6, s.getSuppliedCategory());
81               stmt.setInt(7, s.getSupplierId());
82               return stmt.executeUpdate() > 0;
83           } catch (SQLException e) {
84               throw new DatabaseOperationException("Error updating supplier: " + s.getSupplierId(), e);
85           }
86       }
87
```

Component	Method: update()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 71–87 (17 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–98 Method: delete()

```
88       public boolean delete(int supplierId) {
89           String sql = "DELETE FROM suppliers WHERE supplier_id = ?";
90           try (Connection conn = dbConfig.getConnection();
91                PreparedStatement stmt = conn.prepareStatement(sql)) {
92               stmt.setInt(1, supplierId);
93               return stmt.executeUpdate() > 0;
94           } catch (SQLException e) {
95               throw new DatabaseOperationException("Error deleting supplier: " + supplierId, e);
96           }
97       }
98
```

Component	Method: delete()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 88–98 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 99–112` Method: mapResultSetToSupplier()

```
 99      private Supplier mapResultSetToSupplier(ResultSet rs) throws SQLException {
100          Supplier s = new Supplier();
101          s.setSupplierId(rs.getInt("supplier_id"));
102          s.setName(rs.getString("name"));
103          s.setContactPerson(rs.getString("contact_person"));
104          s.setPhone(rs.getString("phone"));
105          s.setEmail(rs.getString("email"));
106          s.setAddress(rs.getString("address"));
107          s.setSuppliedCategory(rs.getString("supplied_category"));
108          Timestamp ts = rs.getTimestamp("created_at");
109          if (ts != null) s.setCreatedAt(ts.toLocalDateTime());
110          return s;
111      }
112  }
```

Component	Method: mapResultSetToSupplier()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 99–112 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

TableRepository.java

Path: src\main\java\com\restaurant\repository\TableRepository.java | Total Lines: 117 | Logical Blocks: 10

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Data Access Object (DAO) for Table. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–11 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.enums.TableStatus;
5   import com.restaurant.exception.DatabaseOperationException;
6   import com.restaurant.model.RestaurantTable;
7
8   import java.sql.*;
9   import java.util.ArrayList;
10  import java.util.List;
11
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 11. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 12–12 Class Declaration & Heritage

```
12  public class TableRepository {
```

Architecture Role	Defines TableRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 12, allowing cross-package invocation.

Lines 13–14 Class Members & Implementation

```
13      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
14
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 13–14 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 15–30 Method: findById()

```
15      public RestaurantTable findById(int tableId) {
16          String sql = "SELECT * FROM restaurant_tables WHERE table_id = ?";
17          try (Connection conn = dbConfig.getConnection();
18               PreparedStatement stmt = conn.prepareStatement(sql)) {
19              stmt.setInt(1, tableId);
20              try (ResultSet rs = stmt.executeQuery()) {
21                  if (rs.next()) {
22                      return mapResultSetToTable(rs);
23                  }
24              }
25          } catch (SQLException e) {
26              throw new DatabaseOperationException("Error finding table by ID: " + tableId, e);
27          }
28          return null;
29      }
30
```

Component	Method: findById()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 15–30 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–45 Method: findAll()

```
31      public List<RestaurantTable> findAll() {
32          List<RestaurantTable> tables = new ArrayList<>();
33          String sql = "SELECT * FROM restaurant_tables ORDER BY table_number ASC";
34          try (Connection conn = dbConfig.getConnection();
35               PreparedStatement stmt = conn.prepareStatement(sql);
36               ResultSet rs = stmt.executeQuery()) {
37              while (rs.next()) {
38                  tables.add(mapResultSetToTable(rs));
39              }
40          } catch (SQLException e) {
41              throw new DatabaseOperationException("Error fetching tables", e);
42          }
43          return tables;
44      }
45
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 31–45 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 46–69` **Method: create()**

```
46        public boolean create(RestaurantTable table) {
47            String sql = "INSERT INTO restaurant_tables (table_number, capacity, location, status) VALUES
(?, ?, ?, ?)";
48            try (Connection conn = dbConfig.getConnection();
49                 PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
50                stmt.setString(1, table.getTableNumber());
51                stmt.setInt(2, table.getCapacity());
52                stmt.setString(3, table.getLocation());
53                stmt.setString(4, table.getStatus().name());
54
55                int affected = stmt.executeUpdate();
56                if (affected > 0) {
57                    try (ResultSet keys = stmt.getGeneratedKeys()) {
58                        if (keys.next()) {
59                            table.setTableId(keys.getInt(1));
60                        }
61                    }
62                    return true;
63                }
64            } catch (SQLException e) {
65                throw new DatabaseOperationException("Error creating table: " + table.getTableNumber(),
e);
66            }
67            return false;
68        }
69
```

Component	Method: create()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 46–69 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 70–84 Method: update()

```
70       public boolean update(RestaurantTable table) {
71           String sql = "UPDATE restaurant_tables SET table_number = ?, capacity = ?, location = ?, statu
s = ? WHERE table_id = ?";
72           try (Connection conn = dbConfig.getConnection();
73                PreparedStatement stmt = conn.prepareStatement(sql)) {
74               stmt.setString(1, table.getTableNumber());
75               stmt.setInt(2, table.getCapacity());
76               stmt.setString(3, table.getLocation());
77               stmt.setString(4, table.getStatus().name());
78               stmt.setInt(5, table.getTableId());
79               return stmt.executeUpdate() > 0;
80           } catch (SQLException e) {
81               throw new DatabaseOperationException("Error updating table: " + table.getTableId(), e);
82           }
83       }
84
```

Component	Method: update()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 70–84 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 85–96 Method: updateStatus()

```
85       public boolean updateStatus(int tableId, TableStatus status) {
86           String sql = "UPDATE restaurant_tables SET status = ? WHERE table_id = ?";
87           try (Connection conn = dbConfig.getConnection();
88                PreparedStatement stmt = conn.prepareStatement(sql)) {
89               stmt.setString(1, status.name());
90               stmt.setInt(2, tableId);
91               return stmt.executeUpdate() > 0;
92           } catch (SQLException e) {
93               throw new DatabaseOperationException("Error updating status for table: " + tableId, e);
94           }
95       }
96
```

Component	Method: updateStatus()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 85–96 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 97–107` Method: delete()

```
97      public boolean delete(int tableId) {
98          String sql = "DELETE FROM restaurant_tables WHERE table_id = ?";
99          try (Connection conn = dbConfig.getConnection();
100              PreparedStatement stmt = conn.prepareStatement(sql)) {
101             stmt.setInt(1, tableId);
102             return stmt.executeUpdate() > 0;
103         } catch (SQLException e) {
104             throw new DatabaseOperationException("Error deleting table: " + tableId, e);
105         }
106     }
107
```

Component	Method: delete()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 97–107 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 108–117` Method: mapResultSetToTable()

```
108     private RestaurantTable mapResultSetToTable(ResultSet rs) throws SQLException {
109         return new RestaurantTable(
110                 rs.getInt("table_id"),
111                 rs.getString("table_number"),
112                 rs.getInt("capacity"),
113                 rs.getString("location"),
114                 TableStatus.valueOf(rs.getString("status"))
115         );
116     }
117 }
```

Component	Method: mapResultSetToTable()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 108–117 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

UserRepository.java

Path: src\main\java\com\restaurant\repository\UserRepository.java | Total Lines: 146 | Logical Blocks: 10

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Data Access Object (DAO) for User. Implements type-safe SQL queries, transactions, and ResultSet mapping.

Lines 1–11 Package & Library Imports

```
1   package com.restaurant.repository;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.enums.UserRole;
5   import com.restaurant.exception.DatabaseOperationException;
6   import com.restaurant.model.*;
7
8   import java.sql.*;
9   import java.util.ArrayList;
10  import java.util.List;
11
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 11. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 12–12 Class Declaration & Heritage

```
12  public class UserRepository {
```

Architecture Role	Defines UserRepository as a core component in the Restaurant Management System.
Repository / DAO	Data Access Object pattern: translates object-oriented domain models into relational SQL queries.
Access & Modifiers	Declared with public visibility on line 12, allowing cross-package invocation.

Lines 13–14 Class Members & Implementation

```
13      private final DatabaseConfig dbConfig = DatabaseConfig.getInstance();
14
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 13–14 (2 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 15–30 Method: findByUsername()

```
15      public User findByUsername(String username) {
16          String sql = "SELECT * FROM users WHERE username = ? AND is_active = TRUE";
17          try (Connection conn = dbConfig.getConnection();
18               PreparedStatement stmt = conn.prepareStatement(sql)) {
19              stmt.setString(1, username);
20              try (ResultSet rs = stmt.executeQuery()) {
21                  if (rs.next()) {
22                      return mapResultSetToUser(rs);
23                  }
24              }
25          } catch (SQLException e) {
26              throw new DatabaseOperationException("Error fetching user by username: " + username, e);
27          }
28          return null;
29      }
30
```

Component	Method: findByUsername()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 15–30 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–46 Method: findById()

```
31      public User findById(int userId) {
32          String sql = "SELECT * FROM users WHERE user_id = ?";
33          try (Connection conn = dbConfig.getConnection();
34               PreparedStatement stmt = conn.prepareStatement(sql)) {
35              stmt.setInt(1, userId);
36              try (ResultSet rs = stmt.executeQuery()) {
37                  if (rs.next()) {
38                      return mapResultSetToUser(rs);
39                  }
40              }
41          } catch (SQLException e) {
42              throw new DatabaseOperationException("Error fetching user by ID: " + userId, e);
43          }
44          return null;
45      }
46
```

Component	Method: findById()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 31–46 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 47–61` **Method: findAll()**

```
47      public List<User> findAll() {
48          List<User> users = new ArrayList<>();
49          String sql = "SELECT * FROM users ORDER BY user_id ASC";
50          try (Connection conn = dbConfig.getConnection();
51               PreparedStatement stmt = conn.prepareStatement(sql);
52               ResultSet rs = stmt.executeQuery()) {
53              while (rs.next()) {
54                  users.add(mapResultSetToUser(rs));
55              }
56          } catch (SQLException e) {
57              throw new DatabaseOperationException("Error fetching all users", e);
58          }
59          return users;
60      }
61
```

Component	Method: findAll()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 47–61 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 62–90` **Method: create()**

```
62       public boolean create(User user, String passwordHash, String salt) {
63           String sql = "INSERT INTO users (username, password_hash, salt, full_name, email, phone, role,
is_active) " +
64                        "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
65           try (Connection conn = dbConfig.getConnection();
66                PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
67               stmt.setString(1, user.getUsername());
68               stmt.setString(2, passwordHash);
69               stmt.setString(3, salt);
70               stmt.setString(4, user.getFullName());
71               stmt.setString(5, user.getEmail());
72               stmt.setString(6, user.getPhone());
73               stmt.setString(7, user.getRole().name());
74               stmt.setBoolean(8, user.isActive());
75
76               int affected = stmt.executeUpdate();
77               if (affected > 0) {
78                   try (ResultSet keys = stmt.getGeneratedKeys()) {
79                       if (keys.next()) {
80                           user.setUserId(keys.getInt(1));
81                       }
82                   }
83                   return true;
84               }
85           } catch (SQLException e) {
86               throw new DatabaseOperationException("Error creating user: " + user.getUsername(), e);
87           }
88           return false;
89       }
90
```

Component	Method: create()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 62–90 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 91–106 Method: update()

```
 91       public boolean update(User user) {
 92           String sql = "UPDATE users SET full_name = ?, email = ?, phone = ?, role = ?, is_active = ? WH
ERE user_id = ?";
 93           try (Connection conn = dbConfig.getConnection();
 94                PreparedStatement stmt = conn.prepareStatement(sql)) {
 95               stmt.setString(1, user.getFullName());
 96               stmt.setString(2, user.getEmail());
 97               stmt.setString(3, user.getPhone());
 98               stmt.setString(4, user.getRole().name());
 99               stmt.setBoolean(5, user.isActive());
100               stmt.setInt(6, user.getUserId());
101               return stmt.executeUpdate() > 0;
102           } catch (SQLException e) {
103               throw new DatabaseOperationException("Error updating user: " + user.getUserId(), e);
104           }
105       }
106
```

Component	Method: update()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 91–106 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 107–119 Method: updatePassword()

```
107       public boolean updatePassword(int userId, String newHash, String newSalt) {
108           String sql = "UPDATE users SET password_hash = ?, salt = ? WHERE user_id = ?";
109           try (Connection conn = dbConfig.getConnection();
110                PreparedStatement stmt = conn.prepareStatement(sql)) {
111               stmt.setString(1, newHash);
112               stmt.setString(2, newSalt);
113               stmt.setInt(3, userId);
114               return stmt.executeUpdate() > 0;
115           } catch (SQLException e) {
116               throw new DatabaseOperationException("Error updating user password for ID: " + userId, e);
117           }
118       }
119
```

Component	Method: updatePassword()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 107–119 (13 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 120–146 Method: mapResultSetToUser()

```
120       private User mapResultSetToUser(ResultSet rs) throws SQLException {
121           UserRole role = UserRole.valueOf(rs.getString("role"));
122           User user;
123           switch (role) {
124               case ADMIN -> user = new Admin();
125               case MANAGER -> user = new Manager();
126               case WAITER -> user = new Waiter();
127               case CASHIER -> user = new Cashier();
128               case KITCHEN_STAFF -> user = new KitchenStaff();
129               default -> user = new Waiter();
130           }
131           user.setUserId(rs.getInt("user_id"));
132           user.setUsername(rs.getString("username"));
133           user.setPasswordHash(rs.getString("password_hash"));
134           user.setSalt(rs.getString("salt"));
135           user.setFullName(rs.getString("full_name"));
136           user.setEmail(rs.getString("email"));
137           user.setPhone(rs.getString("phone"));
138           user.setRole(role);
139           user.setActive(rs.getBoolean("is_active"));
140           Timestamp ts = rs.getTimestamp("created_at");
141           if (ts != null) {
142               user.setCreatedAt(ts.toLocalDateTime());
143           }
144           return user;
145       }
146   }
```

Component	Method: mapResultSetToUser()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 120–146 (27 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

AppConfig.java

Path: src\main\java\com\restaurant\config\AppConfig.java | Total Lines: 79 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the AppConfig role in the Restaurant Management architecture.

Lines 1–10 Package & Library Imports

```
1  package com.restaurant.config;
2
3  import java.io.InputStream;
4  import java.util.Properties;
5  import java.util.logging.Level;
6  import java.util.logging.Logger;
7
8  /**
9   * Singleton configuration loader that parses application.properties.
10  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
File Handling I/O	File streams and writers (FileWriter, BufferedWriter, PrintWriter) for receipts and log generation.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11  public class AppConfig {
```

Architecture Role	Defines AppConfig as a core component in the Restaurant Management System.
Configuration / Singleton	Manages database properties, JDBC driver registration, and singleton connection management.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–26 Class Members & Implementation

```
12        private static final Logger LOGGER = Logger.getLogger(AppConfig.class.getName());
13        private static AppConfig instance;
14        private final Properties properties = new Properties();
15
16        private AppConfig() {
17            loadProperties();
18        }
19
20        public static synchronized AppConfig getInstance() {
21            if (instance == null) {
22                instance = new AppConfig();
23            }
24            return instance;
25        }
26
```

Component	Class Members & Implementation
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 12–26 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 27–40 Method: loadProperties()

```
27        private void loadProperties() {
28            try (InputStream input = getClass().getClassLoader().getResourceAsStream("config/application.p
roperties")) {
29                if (input != null) {
30                    properties.load(input);
31                } else {
32                    LOGGER.warning("config/application.properties not found on classpath, using fallback d
efaults");
33                    setDefaults();
34                }
35            } catch (Exception e) {
36                LOGGER.log(Level.SEVERE, "Error loading configuration properties", e);
37                setDefaults();
38            }
39        }
40
```

Component	Method: loadProperties()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 27–40 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 41–55 Method: setDefaults()

```
41        private void setDefaults() {
42            properties.setProperty("db.url", "jdbc:mysql://localhost:3306/restaurant_management?useSSL=fal
se&allowPublicKeyRetrieval=true&serverTimezone=UTC");
43            properties.setProperty("db.username", "root");
44            properties.setProperty("db.password", "Pune411019");
45            properties.setProperty("db.driver", "com.mysql.cj.jdbc.Driver");
46            properties.setProperty("app.name", "Grand Feast RMS");
47            properties.setProperty("app.currency", "₹");
48            properties.setProperty("app.taxRate", "5.0");
49            properties.setProperty("app.serviceChargeRate", "2.5");
50            properties.setProperty("restaurant.name", "Grand Feast Fine Dining");
51            properties.setProperty("restaurant.address", "104 Gourmet Boulevard, Silicon City");
52            properties.setProperty("restaurant.phone", "+91 98765 43210");
53            properties.setProperty("restaurant.email", "contact@grandfeast.com");
54        }
55
```

Component	Method: setDefaults()
Line-by-Line Breakdown	Lines 41–55 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 56–59 Method: getProperty()

```
56        public String getProperty(String key) {
57            return properties.getProperty(key);
58        }
59
```

Component	Method: getProperty()
Line-by-Line Breakdown	Lines 56–59 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 60–63 Method: getProperty()

```
60        public String getProperty(String key, String defaultValue) {
61            return properties.getProperty(key, defaultValue);
62        }
63
```

Component	Method: getProperty()
Line-by-Line Breakdown	Lines 60–63 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 64–71 Method: getDouble()

```
64      public double getDouble(String key, double defaultValue) {
65          try {
66              return Double.parseDouble(properties.getProperty(key));
67          } catch (Exception e) {
68              return defaultValue;
69          }
70      }
71
```

Component	Method: getDouble()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 64–71 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–79 Method: getInt()

```
72      public int getInt(String key, int defaultValue) {
73          try {
74              return Integer.parseInt(properties.getProperty(key));
75          } catch (Exception e) {
76              return defaultValue;
77          }
78      }
79  }
```

Component	Method: getInt()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 72–79 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

DatabaseConfig.java

Path: src\main\java\com\restaurant\config\DatabaseConfig.java | Total Lines: 83 | Logical Blocks: 8

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the DatabaseConfig role in the Restaurant Management architecture.

Lines 1–13 Package & Library Imports

```
1   package com.restaurant.config;
2
3   import com.restaurant.exception.DatabaseOperationException;
4
5   import java.sql.Connection;
6   import java.sql.DriverManager;
7   import java.sql.SQLException;
8   import java.util.logging.Level;
9   import java.util.logging.Logger;
10
11  /**
12   * Thread-safe Database Configuration and Connection Factory.
13   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Concurrency Support	Multi-threading utilities (BlockingQueue, Executors, Atomic primitives) used for asynchronous tasks.
Line Analysis	Covers lines 1 to 13. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 14–14 Class Declaration & Heritage

```
14  public class DatabaseConfig {
```

Architecture Role	Defines DatabaseConfig as a core component in the Restaurant Management System.
Configuration / Singleton	Manages database properties, JDBC driver registration, and singleton connection management.
Access & Modifiers	Declared with public visibility on line 14, allowing cross-package invocation.

Lines 15–37 Class Members & Implementation

```
15      private static final Logger LOGGER = Logger.getLogger(DatabaseConfig.class.getName());
16      private static volatile DatabaseConfig instance;
17      private final String url;
18      private final String username;
19      private final String password;
20      private final String driver;
21
22      private DatabaseConfig() {
23          AppConfig appConfig = AppConfig.getInstance();
24          this.url = appConfig.getProperty("db.url");
25          this.username = appConfig.getProperty("db.username");
26          this.password = appConfig.getProperty("db.password");
27          this.driver = appConfig.getProperty("db.driver", "com.mysql.cj.jdbc.Driver");
28
29          try {
30              Class.forName(driver);
31              LOGGER.info("MySQL JDBC Driver registered successfully.");
32          } catch (ClassNotFoundException e) {
33              LOGGER.log(Level.SEVERE, "MySQL JDBC Driver not found in classpath!", e);
34              throw new DatabaseOperationException("JDBC Driver class not found: " + driver, e);
35          }
36      }
37
```

Component	Class Members & Implementation
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 15–37 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 38–53 Method: getInstance()

```
38      public static DatabaseConfig getInstance() {
39          if (instance == null) {
40              synchronized (DatabaseConfig.class) {
41                  if (instance == null) {
42                      instance = new DatabaseConfig();
43                  }
44              }
45          }
46          return instance;
47      }
48
49      /**
50       * Establishes and returns a fresh JDBC Connection.
51       * @return active java.sql.Connection
52       * @throws DatabaseOperationException if connection cannot be opened
53       */
```

Component	Method: getInstance()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 38–53 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 54–66 Method: getConnection()

```
54      public Connection getConnection() {
55          try {
56              return DriverManager.getConnection(url, username, password);
57          } catch (SQLException e) {
58              LOGGER.log(Level.SEVERE, "Failed to connect to MySQL database at: " + url, e);
59              throw new DatabaseOperationException("Database connection failed. Please ensure MySQL is r
unning.", e);
60          }
61      }
62
63      /**
64       * Diagnostic method to test connectivity.
65       * @return true if connection succeeded and valid, false otherwise.
66       */
```

Component	Method: getConnection()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 54–66 (13 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 67–75 Method: testConnection()

```
67      public boolean testConnection() {
68          try (Connection conn = getConnection()) {
69              return conn != null && !conn.isClosed();
70          } catch (Exception e) {
71              LOGGER.warning("Database connectivity test failed: " + e.getMessage());
72              return false;
73          }
74      }
75
```

Component	Method: testConnection()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 67–75 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 76–79 Method: getUrl()

```
76      public String getUrl() {
77          return url;
78      }
79
```

Component	Method: getUrl()
Line-by-Line Breakdown	Lines 76–79 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 80–83 Method: getUsername()

```
80      public String getUsername() {
81          return username;
82      }
83  }
```

Component	Method: getUsername()
Line-by-Line Breakdown	Lines 80–83 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.